Experiment No. 1

Title

Introduction to Database Creation Using MySQL / PostgreSQL

Aim

To create a Student Course Management System database and establish relationships between tables.

Prerequisite

- Basic knowledge of DBMS concepts
- Understanding of tables, primary keys, foreign keys
- Basic SQL syntax

Hardware Requirement

- Computer with minimum 4GB RAM
- Internet connection

Software Requirement

- SQL++ / PostgreSQL
- MySQL Workbench / pgAdmin
- Windows / Linux OS

Theory

A **Database** is an organized collection of related data.
A **Relational Database** stores data in tables connected using relationships.

Entities:

1. **Student**
2. **Course**
3. **Instructor**
4. **Enrollment**

Relationships:

- One Student → Many Courses (Many-to-Many via Enrollment)
- One Course → Many Students

- One Instructor → Many Courses

Primary Key: Uniquely identifies a record
Foreign Key: Creates relationship between tables

Steps

Step 1: Install Database Software

- Install MySQL / PostgreSQL
- Open MySQL Workbench or pgAdmin

Step 2: Create Database

```
CREATE DATABASE StudentDB;
USE StudentDB;
```

Purpose:
Creates a new database environment.

Step 3: Create Student Table

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Email VARCHAR(50),
    Age INT,
    Address VARCHAR(100)
);
```

Explanation:

- INT → numeric value
- VARCHAR → variable character string
- PRIMARY KEY → unique identifier

Step 4: Create Instructor Table

```
CREATE TABLE Instructor (
    InstructorID INT PRIMARY KEY,
```

```
    Name VARCHAR(50),
    Email VARCHAR(50),
    Department VARCHAR(50)
);
```

Step 5: Create Course Table

```
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50),
    Credits INT,
    InstructorID INT,
    FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID)
);
```

Purpose:
Creates relationship between Course and Instructor.

Step 6: Create Enrollment Table

```
CREATE TABLE Enrollment (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

Purpose:
Implements many-to-many relationship.

Step 7: Insert Sample Data

```
INSERT INTO Student VALUES (1,'Ali','ali@gmail.com',21,'Pune');
```

Repeat for other tables.

Step 8: Verify Tables

```
SHOW TABLES;
DESC Student;
SELECT * FROM Student;
```

Conclusion

The Student Course Management System database was successfully created with proper relationships using primary and foreign keys.

Assignments

1. Explain key with example.
2. What is a ER relationship?
3. Demonstrate Data Base management System .
4. Illustrate Advantages of Data Base management System.

.

Experiment No. 2

Title

Views and Indexing in Student Course Management System

Aim

To create views and use indexing to improve query performance.

Prerequisite

- Knowledge of SELECT queries
- Understanding of indexing concept

Hardware & Software Requirement

Hardware Requirement

- Computer with minimum 4GB RAM
- Internet connection

Software Requirement

- SQL++ / PostgreSQL
- MySQL Workbench
- Windows / Linux OS

Theory

View:

A **View** is a virtual table based on SQL query. It does not store data physically.

A **View** is a virtual table created using SELECT query.

It:

- Does not store data physically
- Shows data from one or more tables
- Simplifies complex queries

Advantages of View

- Data abstraction
- Security (hide sensitive columns)
- Reusability
- Simplified reporting

2. Index

An **Index** is a database object that improves query performance.

Without Index:

- Full table scan

With Index:

- Faster search using sorted structure (B-Tree)

Types of Index

- Primary Index
- Unique Index

Advantages:

- Security
- Simplifies complex queries
- Easy reporting

Index:

An **Index** improves speed of data retrieval.

Steps

Step 1: Create View

```
CREATE VIEW StudentCourseView AS
SELECT s.StudentID, s.Name, c.CourseName, c.Credits
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Course c ON e.CourseID = c.CourseID;
```

Step 2: Display View

```
SELECT * FROM StudentCourseView;
```

Step 3: Update Through View

```
UPDATE Student
SET Age = 22
WHERE StudentID = 1;
```

Then check view.

Step 4: Create Index

```
CREATE INDEX idx_name
ON Student(Name);
```

Step 5: Measure Performance

Before index:

```
EXPLAIN SELECT * FROM Student WHERE Name='Ali';
```

Conclusion

Views simplify data access and indexing significantly improves query performance.

Assignment

1. Explain view?
2. Illustrate advantages of view.
3. State, can we update a view?
4. Demostrate indexing and advantages of indexing?
5. Illustrate type of indexes?

Experiment No. 3

Title

SQL Queries with Filters, Sorting and Nested Sub queries

Aim

To perform advanced SQL queries using WHERE, ORDER BY, GROUP BY and Subqueries.

Prerequisite

- Basic knowledge of DBMS concepts
- Understanding of tables, primary keys, foreign keys
- Basic SQL syntax

Hardware Requirement

- Computer with minimum 4GB RAM
- Internet connection

Software Requirement

- MySQL / PostgreSQL
- MySQL Workbench / pgAdmin
- Windows / Linux OS

Theory

WHERE Clause

Used to filter records.

ORDER BY

Used to sort records.

GROUP BY

Used with aggregate functions.

Subquery

Query inside another query.

. WHERE Clause

Filters records based on condition.

Operators:

- =, >, <, >=
- AND, OR
- BETWEEN
- LIKE
- IN

2. ORDER BY

Sorts data:

- ASC (Ascending)
- DESC (Descending)

3. GROUP BY

Used with:

- COUNT()
- AVG()
- SUM()
- MIN()
- MAX()

4. HAVING Clause

Used to filter aggregated results.

5. Subquery

A query inside another query.

Types:

- Single-row
- Multi-row
- Correlated

Steps

Step 1: Filtering

```
SELECT * FROM Student
WHERE Age > 20;
```

Step 2: Sorting

```
SELECT * FROM Student
ORDER BY Name ASC;
```

Step 3: Aggregation

```
SELECT CourseID, COUNT(StudentID)
FROM Enrollment
GROUP BY CourseID;
```

Step 4: HAVING

```
SELECT CourseID, COUNT(StudentID)
FROM Enrollment
GROUP BY CourseID
HAVING COUNT(StudentID) > 2;
```

Step 5: Nested Subquery

```
SELECT Name FROM Student
WHERE StudentID IN
(SELECT StudentID FROM Enrollment
 WHERE CourseID =
 (SELECT CourseID FROM Course
  WHERE CourseName='DBMS'));
```

1.

Conclusion

Advanced SQL queries help retrieve filtered, sorted and summarized data efficiently.

Assignment

1. Illustrtea purpose of where clause?
2. Compar beetween ORDER BY ASC and DESC.
3. Illustrate is subquery?
4. Explain GROUP BY with an example?
5. Compare between aggregate and normal function?